

Documentation du projet Pac-Man

Cours : Programmation Orientée Objet (POO) & Méthodologie des SI

Intervenants : Maxime Harlé & Antonin Kalk

Membre de l'équipe : LIS Ambre, EL KAID Karim, LI Haoxuan, LAAZIBI Imane

Table des matières

Projet	3
Les fonctionnalités	3
Les apports supplémentaires par rapport au jeu de base	3
Déroulement du projet.....	4
Organisation	4
Difficultés	5
Solutions	6

1. Le projet

a. Les fonctionnalités

Le projet reproduit le jeu Pac-Man original. Le joueur contrôle Pac-Man à l'aide des touches directionnelles et doit collecter l'ensemble des pastilles disposées dans le labyrinthe tout en évitant les fantômes.

Le labyrinthe est construit à partir d'une Tilemap et les murs bloquent aussi bien le joueur que les fantômes. Des pastilles classiques sont réparties dans les couloirs et rapportent des points à chaque collecte. Des super-pastilles sont aussi placées sur la carte et permettent à Pac-Man de devenir temporairement invincible et de manger les fantômes.

Quatre fantômes sont présents sur la carte chacun avec un comportement distinct. Ils démarrent en prison et sont relâchés progressivement. Si le joueur entre en collision avec un fantôme hors état de vulnérabilité il perd une vie.

Le système de déplacement des fantômes repose sur un mécanisme de nodes. Le labyrinthe contient des points de décision invisibles placés aux intersections et bifurcations des couloirs. Quand un fantôme atteint un node il évalue les directions disponibles et choisit la suivante selon son comportement. Ce système permet de contrôler précisément les déplacements sans analyser en permanence l'ensemble de la carte.

L'interface affiche en permanence le score actuel le round en cours et le nombre de vies restantes.

Au lancement du jeu une barre de chargement s'affiche puis trois boutons apparaissent :

- Jouer sur la map 1
- Jouer sur la map 2
- Créer une map depuis l'éditeur

Un écran de Game Over s'affiche quand toutes les vies sont épuisées et propose de recommencer.

b. Les apports supplémentaires par rapport au jeu de base

Nous avons intégré plusieurs fonctionnalités allant au-delà du jeu de base correspondant aux différents checkpoints du sujet.

Pour la gestion de la difficulté un menu de paramétrage est disponible avant le lancement d'une partie. Il permet de régler la vitesse de Pac-Man et des fantômes la durée d'effet de la super-pastille et le temps de détention des fantômes en prison. Ce

même menu permet de régler le nombre de fantômes dans la partie allant de 1 à 4 en choisissant lesquels seront présents parmi Blinky, Pinky, Inky et Clyde.

Deux labyrinthes distincts sont disponibles et sélectionnables depuis le menu principal.

Nous avons implémenté quatre fantômes chacun avec un comportement de chasse différent :

- Le fantôme Chase (Blinky) se dirige à chaque node vers la direction qui le rapproche le plus de la position actuelle de Pac-Man. C'est le comportement le plus direct et le plus agressif.
- Le fantôme Ambusher (Inky) ne cible pas la position actuelle de Pac-Man mais un point situé quatre cases devant lui dans sa direction de déplacement
- Le fantôme Flanker (Clyde) calcule un point pivot situé deux cases devant Pac-Man puis applique une symétrie par rapport à la position du fantôme Chase pour obtenir sa destination. Ce comportement crée un effet de prise en tenaille avec Blinky.
- Le fantôme Boo (Pinky) adopte un comportement imprévisible. À chaque frame il calcule si Pac-Man se déplace dans sa direction. Si c'est le cas Boo inverse sa logique et fuit au lieu de poursuivre. Dès que Pac-Man change de trajectoire Boo reprend une approche normale.

Nous avons aussi intégré un éditeur de map permettant de concevoir ses propres labyrinthes directement depuis le jeu. Les maps créées peuvent être sauvegardées et rechargées.

Enfin, nous avons implémenté une IA avec ML-Agents de Unity capable de jouer automatiquement à la place du joueur. L'agent observe l'état du jeu (position, murs, fantômes et pastilles) puis choisit une direction parmi les quatre possibles. Il est récompensé lorsqu'il collecte des pastilles, mange des fantômes ou termine un round, et pénalisé lorsqu'il se fait attraper. Après un entraînement sur une scène dédiée avec accélération du temps, le modèle peut être activé en jeu avec la touche P pour remplacer le contrôle du joueur.

2. Déroutement du projet

a. Organisation

Pour débiter le projet chaque membre de l'équipe a suivi une phase d'auto-formation sur Unity adaptée à son niveau. Cette montée en compétences s'est faite via des tutoriels vidéo des forums et une exploration personnelle du logiciel.

En parallèle de cette formation nous avons produit plusieurs diagrammes UML : un diagramme d'activité un diagramme d'états-transitions pour Pac-Man et pour les fantômes un diagramme de cas d'utilisation et un diagramme de classes. Cette étape était importante car au démarrage du projet beaucoup de choses restaient floues et ces diagrammes nous ont permis de partir sur une base commune.

Certains membres ne disposant pas d'expérience avec Unity nous avons choisi de nous appuyer sur un tutoriel portant sur la réalisation d'un jeu Pac-Man sous Unity pour avoir une base solide. Ce tutoriel nous a permis de découper le projet en quatre domaines : la map le game manager Pac-Man et les fantômes. Chaque membre s'est vu attribuer un de ces domaines et le travail a été organisé en tâches à réaliser entre chaque séance.

Exemple de répartition :

Jour	P1 - Niveau	P2 - Pac-Man	P3 - Fantômes	P4 - Game Manager
J1	● Tilemap maze (début)	● Préfab Pac-Man ● Mouvement base	● Préfabs fantômes ● Mouvement base	● GameManager base
J2	● Tilemap maze complet ● Colliders murs	● Peut tester (demi-niveau) ● Input + animations	● Peut tester (demi-niveau) ● Script mouvement	● Système rounds
J3 - MERGE #1	● MERGE niveau ● Tests intégration	● PULL & TEST ● Vérifie maze	● PULL & TEST ● Vérifie maze	● REVIEW ● Valide le merge
J4	● Placement pellets	● Collision pellets ● Tests dans vrai maze	● Tests mouvement maze ■ Attend nœuds pour IA	● Score système
J5	● Nœuds placement complet ● Power pellets	● Consommation pellets ■ Power pellets pas prêts	■ P1 DÉBLOQUE P3 ● Mode Chase	● Gestion vies
J6 - MERGE #2	● MERGE pellets/nœuds ● Tests	● MERGE Pac-Man ● Tests collision	● MERGE fantômes base ● Tests mouvement	● REVIEW globale ● Test jeu jouable

À chaque séance nous mergions les branches Git et planifiions les avancées suivantes ce qui nous a permis de progresser de façon itérative séance après séance.

Une fois une version stable obtenue nous nous sommes consacrés aux fonctionnalités complémentaires demandées dans le sujet en gardant le même principe de répartition équitable des tâches.

Sachant que le projet s'appuyait sur un tutoriel nous avons voulu nous l'approprier davantage. Nous avons donc entrepris une phase de refactorisation pour produire un code plus personnel et mieux structuré. Cela a introduit des interfaces supplémentaires un découpage plus fin des fonctions et une personnalisation générale du code.

b. Difficultés

La première difficulté a été la prise en main d'Unity. Nous avions des niveaux de départ hétérogènes ce qui a demandé un effort de mise à niveau collectif. Le logiciel peut aussi s'avérer complexe à maîtriser et nous avons rencontré des instabilités liées aux différences de versions entre membres malgré une version commune définie en début de projet.

La gestion des branches Git a aussi posé des problèmes surtout lors des premières fusions. Nous travaillions tous sur la même scène Unity et des conflits de merge

apparaissaient régulièrement car Unity modifie automatiquement certains fichiers de scène de façon non prévisible. Il était alors difficile de savoir quoi conserver.

La refactorisation a elle aussi rencontré des obstacles. Nous avons commencé par supprimer les corps de fonctions existants avant de les réécrire ce qui a rendu le projet temporairement non fonctionnel et créé des incompréhensions au sein de l'équipe. Avancer sur un code partiellement désactivé sans communication constante s'est révélé vraiment peu efficace.

c. Solutions

Pour homogénéiser nos niveaux nous avons organisé la phase d'auto-formation en parallèle de la conception des diagrammes UML pour que chacun aborde le développement avec un socle de connaissances comparable.

Les problèmes de version ou de crash Unity ont été réglés de façon individuelle car ces incidents étaient souvent isolés et ne nécessitaient pas d'intervention collective.

Pour les conflits de scène nous avons cherché des solutions via des forums des ressources en ligne et des outils d'IA. Cela nous a permis de merger l'ensemble des branches et de rétablir un projet cohérent. Par la suite chaque membre a travaillé sur sa propre scène Unity ce qui a éliminé les conflits récurrents.

Pour la refactorisation nous avons changé d'approche. Nous avons établi une répartition précise des responsabilités défini les modifications collectivement en amont et appliqué les changements de façon progressive en gardant le code existant comme base. Cela a permis à chacun de comprendre ce que les autres faisaient tout au long du processus.